QUANTUM FIBER / TECHNICAL FIELD GUIDE

C6500XK firmware backup procedure

A complete, read-only NAND extraction workflow, based on a successful root-shell backup on 7 November 2026.

CONFIRMED RESULT

11 of 11 MTD partitions saved to a 32 GB USB drive. All 11 SHA-256 checks returned OK. The modem continued operating, and optics were reported Up/GOOD. The raw images stayed private.

IMPORTANT LIMITS

This procedure requires existing root-shell access. It does not explain how to obtain root access. The system UBI image was captured live, so it is not a guaranteed consistent restore image. Do not flash these files back without a separate restore plan.

Quick reference

Device	Confirmed configuration
Product	Quantum Fiber / Gemtek C6500XK SmartNID
Processor	EcoNet EN7580, MIPS32 interAptiv, 4 logical CPUs
Kernel	Linux 4.4.115, build dated 30 Oct 2024
Flash	NAND MTD, 256 KiB eraseblocks, 4 KiB pages, 128-byte OOB
Live root	SquashFS at /rom, writable overlay on UBI/UBIFS
USB backup	/dev/sda1, FAT, mounted at /mnt/sda1
Backup folder	/mnt/sda1/C6500XK-backup/

Use the manual steps in Sections 1-8. Appendix A includes a reusable shell script that follows the same extraction methods. The script is supplied as a convenience and has not been executed on your hardware by this document creator.

1. Rules before you begin

- Use a root shell on the C6500XK. Do not change the flash contents.
- Keep the modem powered. Avoid rebooting it while reading NAND.
- Use only the verified USB mount. Never assume that /dev/sda belongs to the USB drive.

- Use read-only flash inputs such as /dev/mtdNro for direct reads.
- Keep incomplete files and logs for diagnosis. Do not rename a partial file as a completed image.
- Keep backup images private. They may contain certificates, keys, serial numbers, and configuration data.

2. Prepare the USB drive

Use a dedicated USB flash drive. The successful backup used a 32.09 GB drive (29.88 GiB). A 4 GB drive has enough capacity for these files, but 16 GB or larger provides a safer margin.

2.1 Format on Windows, before connecting to the modem

- Back up any existing data. Formatting removes files from the selected USB partition.
- Choose an MBR partition layout and FAT32 filesystem. Use the default allocation unit size.
- Use the volume label C6500BACKUP. Select Quick Format.
- If Windows does not offer FAT32, use a reliable FAT32 formatting utility. Do not format any disk inside the modem.
- Eject the USB drive from Windows. Insert it into the C6500XK USB port.

The tested drive had /dev/sda1 as its main FAT partition. Another 1 MiB partition was visible. No change to that small partition was needed.

2.2 Identify the drive and mount

```
ls /sys/bus/usb/devices/
cat /proc/partitions
ls -l /dev/sd* 2>/dev/null
cat /proc/filesystems
cat /proc/mounts | grep ' /mnt/sda1 '
df -h /mnt/sda1
```

The proven setup automatically mounted **/dev/sda1** at **/mnt/sda1**. Confirm that the path is mounted before writing anything. Seeing a directory alone is not enough.

If the drive is detected as **/dev/sda1**, FAT/VFAT is supported, and it is not mounted, a manual mount can be attempted:

```
mkdir -p /mnt/sda1
mount -t vfat /dev/sda1 /mnt/sda1
cat /proc/mounts | grep ' /mnt/sda1 '
```

> **STOP IF THE DEVICE DIFFERS**
>
> If the device name, partition, or mounted filesystem differs, identify it first. Do not run mount, format, or dump commands against an unverified storage device.

3. Record the device and flash layout

Run the following commands to confirm the device. They read system information only.

```
id -u
uname -a
cat /proc/cpuinfo
cat /proc/cmdline
cat /proc/mtd
cat /proc/partitions
ls -l /dev/mtd* 2>/dev/null
cat /proc/mounts
cat /sys/class/mtd/mtd8/type
```

In the verified session, **id -u** returned 0, **/proc/cmdline** contained **ubi.mtd=system**, and mtd8 was NAND flash. The MTD partition map was:

MTD	Name	Size (bytes)	Successful method
0	bootloader	2,097,152	nanddump
1	dsd	2,097,152	nanddump
2	kernel	2,590,920	dd /dev/mtd2ro
3	rootfs	22,478,848	dd /dev/mtd3ro
4	tclinux	67,108,864	nanddump
5	kernel_slave	2,583,253	dd /dev/mtd5ro
6	rootfs_slave	36,503,552	dd /dev/mtd6ro
7	tclinux_slave	67,108,864	nanddump
8	system	335,544,320	nanddump
9	uenv	2,097,152	nanddump
10	art	1,572,864	nanddump

Total of 11 saved file sizes: 541,782,941 bytes (516.68 MiB). Some firmware partitions may describe overlapping image regions. Do not use the sum to infer physical chip capacity.

3.1 Save inventory and layout reports

```
OUT=/mnt/sda1/C6500XK-backup
mkdir -p "$OUT"
cat /proc/mtd > "$OUT/partition-map.txt"
uname -a > "$OUT/kernel-version.txt"
cat /proc/cmdline > "$OUT/kernel-cmdline.txt"
cat /proc/mounts > "$OUT/mounted-filesystems.txt"
mtdinfo -a > "$OUT/mtd-info.txt" 2>&1
ubinfo -a > "$OUT/ubi-info.txt" 2>&1
sync
```

The successful device showed three UBI volumes inside the system partition: **rootfs_data**, **config**, and **log**. Their reported data sizes were 33,775,616, 17,014,784, and 17,014,784 bytes.

4. Confirm read tools and NAND properties

```
for x in nanddump mtdinfo ubinfo dd tar scp busybox; do
    echo "=== $x ==="
    command -v "$x" || echo "Not found"
done
nanddump --help 2>&1 | grep -E 'padbad|omitoob'
mtdinfo -a
```

The proven device had **/usr/sbin/nanddump**, **/usr/sbin/mtdinfo**, **/usr/sbin/ubinfo**, **/bin/dd**, **/bin/tar**, and **/usr/bin/scp**. It accepted both **--bb=padbad** and **--omitoob**.

Meaning: --bb=padbad preserves logical positions when bad blocks are padded. --omitoob leaves NAND spare-area bytes out of the file. The output is not a physical OOB-inclusive chip image.

5. Make the first two NAND backups

5.1 mtd0: bootloader

```
OUT=/mnt/sda1/C6500XK-backup
nanddump --bb=padbad --omitoob \
  -f "$OUT/mtd0_bootloader.bin" /dev/mtd0
wc -c "$OUT/mtd0_bootloader.bin"
```

The verified output was exactly **2,097,152 bytes**. The diagnostic report showed zero ECC failures, zero corrections, and zero bad blocks.

5.2 mtd1: dsd

```
nanddump --bb=padbad --omitoob \
  -f "$OUT/mtd1_dsd.bin" /dev/mtd1
wc -c "$OUT/mtd1_dsd.bin"
```

This image was also exactly **2,097,152 bytes**. No ECC failures or corrections were reported.

6. The mtd2 error and its tested recovery

> **OBSERVED ERROR**
>
> nanddump exited with status 1 while checking the last partial eraseblock of mtd2. The error text was: `bad eraseblock number 9, mtd2 has 9 eraseblocks`. The first attempt left `mtd2_kernel.bin.partial`.

The kernel partition has **2,590,920 bytes**. Nine complete 262,144-byte eraseblocks hold 2,359,296 bytes. The final **231,624 bytes** form a partial tenth eraseblock. On this system, nanddump did not handle that trailing region.

The four partitions with sizes not aligned to the eraseblock are **mtd2, mtd3, mtd5, and mtd6**. These were successfully read with dd from the read-only MTD character devices.

6.1 Preserve the failed file and recover mtd2

```
OUT=/mnt/sda1/C6500XK-backup
# Do not delete mtd2_kernel.bin.partial.
dd if=/dev/mtd2ro \
  of="$OUT/mtd2_kernel.bin.dd.partial" bs=4096
cat /sys/class/mtd/mtd2/size
wc -c "$OUT/mtd2_kernel.bin.dd.partial"
```

The successful dd run returned zero exit errors and produced **2,590,920 bytes**. After the size check, the recovered image was saved as **mtd2_kernel.bin**. The failed nanddump partial remained available for diagnostics.

```
# Only after dd succeeds and both sizes match:
mv "$OUT/mtd2_kernel.bin.dd.partial" \
   "$OUT/mtd2_kernel.bin"
```

6.2 Read the three other unaligned partitions

```
dd if=/dev/mtd3ro of="$OUT/mtd3_rootfs.bin" bs=4096
dd if=/dev/mtd5ro of="$OUT/mtd5_kernel_slave.bin" bs=4096
dd if=/dev/mtd6ro of="$OUT/mtd6_rootfs_slave.bin" bs=4096
```

Check every exit status and every file size against **/sys/class/mtd/mtdN/size**. In a new run, use temporary output names first. Rename each file only after a successful read and exact size check.

These dd reads use kernel-managed MTD reads. They do not include NAND OOB bytes. They also do not produce the same detailed per-read ECC report that nanddump printed.

7. Dump the remaining eraseblock-aligned partitions

Use nanddump for the remaining partitions. This was the successful method for mtd4, mtd7, mtd8, mtd9, and mtd10.

```
OUT=/mnt/sda1/C6500XK-backup
for i in 4 7 8 9 10; do
    NAME=$(cat /sys/class/mtd/mtd$i/name)
    FILE="$OUT/mtd${i}_${NAME}.bin"
    nanddump --bb=padbad --omitoob \
      -f "$FILE" "/dev/mtd$i" || break
done
sync
```

The completed nanddump operations reported **zero ECC failures, zero ECC corrections, and zero bad blocks** in their diagnostics. Stop and investigate any nonzero exit status, ECC error, or unexpected size.

> **LIVE UBI WARNING**
>
> The 320 MiB mtd8 system image was read while its UBI volumes were mounted and active. Its SHA-256 checksum verifies the saved bytes. It does not prove a consistent filesystem snapshot or a safe restoration image.

8. Verify all 11 images and save checksums

First check the file count, file sizes, partition map, and UBI layout. Then create and check SHA-256 hashes.

```
cd /mnt/sda1/C6500XK-backup
ls -lh mtd*.bin
ls mtd*.bin | wc -l
for i in 0 1 2 3 4 5 6 7 8 9 10; do
    NAME=$(cat /sys/class/mtd/mtd$i/name)
    FILE="mtd${i}_${NAME}.bin"
    EXPECTED=$(cat /sys/class/mtd/mtd$i/size)
    ACTUAL=$(wc -c < "$FILE")
    echo "mtd$i: expected=$EXPECTED actual=$ACTUAL"
done
sha256sum mtd*.bin > SHA256SUMS
sha256sum -c SHA256SUMS
sync
```

The completed session had **11 files** and **11 SHA-256 results marked OK**. SHA-256 confirms that saved files match the hashes written to SHA256SUMS. It cannot prove a perfect NAND read or live filesystem consistency.

8.1 Verification checklist

- The 11 completed files exist, with the names shown in Section 3.
- Every file size matches the corresponding sysfs size.
- SHA256SUMS exists and all 11 checks return OK.
- partition-map.txt, mtd-info.txt, and ubi-info.txt are saved.
- Logs and any failed .partial images remain available separately.
- The final sync command has finished.

9. Restore normal operation and remove USB safely

If you changed any maintenance settings to gain access, restore those settings and verify normal operation. The successful session confirmed optics reported **Up/GOOD**.

Exit the USB directory before unmounting it. A shell with its current directory on the drive causes **Resource busy**. This exact issue occurred in the successful session.

```
cd /
sync
umount /mnt/sda1
mount | grep sda1
```

If umount succeeds and **mount | grep sda1** prints nothing, remove the USB stick. If unmount reports Resource busy, do not unplug it. Check for open shell directories or processes that still use the drive.

10. Copy and examine the backup offline

Copy the entire **C6500XK-backup** folder to a private directory on Windows. Keep the original USB contents unchanged. Verify the copied hashes in WSL or Linux before inspecting any images.

```
cd /path/to/C6500XK-backup
sha256sum -c SHA256SUMS
file mtd2_kernel.bin mtd3_rootfs.bin
file mtd5_kernel_slave.bin mtd6_rootfs_slave.bin
# Optional, if binwalk is installed:
binwalk mtd2_kernel.bin
binwalk mtd3_rootfs.bin
binwalk mtd4_tclinux.bin
```

Use a non-root Linux account for ordinary offline analysis. The next investigation can identify the EcoNet PON drivers, ONU configuration, image formats, and primary/secondary firmware differences.

> **SECURITY AND RESTORE LIMITATION**
>
> Treat the raw images as secrets. Do not upload the full system, config, bootloader, or calibration partitions publicly. A verified backup is not automatically a bootable or safe-to-flash recovery image.

11. Troubleshooting and decision table

Condition	Recommended action
USB directory exists, but is not mounted	Stop. Check /proc/mounts and identify the actual USB partition.
nanddump fails on an unaligned MTD partition	Keep the partial file. Read the corresponding /dev/mtdNro device with dd; verify its size.
nanddump reports a bad block or ECC error	Stop and review the diagnostic log. Do not assume a readable, restorable image.
dd exits nonzero or the output is short	Keep the incomplete file. Investigate before continuing.
File size differs from sysfs size	Mark the backup incomplete. Do not include it in the verified set.
Hash verification fails after transfer	Re-copy the file from the original USB backup. Preserve the original.
umount reports Resource busy	Run cd /, close processes using the USB path, then retry sync and umount.

Appendix A. Reusable full backup script

This companion script uses the same proven methods. It reads all 11 partitions, skips existing files only when their sizes match, saves diagnostics, records layout reports, checks SHA-256 hashes, and stops on failure. It does not erase or write NAND.

Save the script as **backup-c6500xk.sh**. Place it on the mounted USB drive, then run **sh /mnt/sda1/backup-c6500xk.sh** from the modem root shell. Review the script and confirm the USB mount first. This reusable wrapper is reconstructed from the session; it has not itself been run on the modem.

> **ABOUT EXISTING PARTIAL FILES**
>
> The script writes new temporary files with .work suffixes. It never deletes the previously failed mtd2_kernel.bin.partial. A completed image is skipped only if its size matches sysfs.

Script lines 1-33 of 86

```
#!/bin/sh
# C6500XK read-only NAND backup to an already-mounted FAT32 USB drive.
# Reconstructed from a successful 2026-11-07 backup session.
# Does not write to MTD devices. Run as root after checking the target mount.
set -eu

USB=/mnt/sda1
OUT="$USB/C6500XK-backup"

if ! grep -q ' /mnt/sda1 ' /proc/mounts; then
    echo "ERROR: $USB is not a mounted filesystem. Stop."
    exit 1
fi
if [ "$(id -u)" != 0 ]; then
    echo "ERROR: A root shell is required to read the MTD devices."
    exit 1
fi
if ! command -v nanddump >/dev/null 2>&1 || ! command -v dd >/dev/null 2>&1; then
    echo "ERROR: nanddump and dd must be available."
    exit 1
fi

mkdir -p "$OUT"

# Preserve the observed NAND/UBI layout and running environment.
cat /proc/mtd > "$OUT/partition-map.txt"
uname -a > "$OUT/kernel-version.txt"
cat /proc/cmdline > "$OUT/kernel-cmdline.txt"
cat /proc/mounts > "$OUT/mounted-filesystems.txt"
mtdinfo -a > "$OUT/mtd-info.txt" 2>&1 || echo 'WARNING: mtdinfo metadata failed.'
ubinfo -a > "$OUT/ubi-info.txt" 2>&1 || echo 'WARNING: ubinfo metadata failed.'

# mtd2, mtd3, mtd5, and mtd6 have partial eraseblocks.
```

Script lines 34-66 of 86

```
# nanddump --bb=padbad fails on mtd2 with this firmware's partition sizes.
# Direct reads through /dev/mtdNro were successful for these four partitions.
for i in 0 1 2 3 4 5 6 7 8 9 10; do
    NAME=$(cat "/sys/class/mtd/mtd$i/name")
    EXPECTED=$(cat "/sys/class/mtd/mtd$i/size")
    FILE="$OUT/mtd${i}_${NAME}.bin"
    WORK="$FILE.work"
    LOG="$OUT/mtd${i}_${NAME}.dump.log"

    if [ -f "$FILE" ]; then
        ACTUAL=$(wc -c < "$FILE")
        if [ "$ACTUAL" -ne "$EXPECTED" ]; then
            echo "ERROR: Existing file has wrong size: $FILE"
            exit 1
        fi
        echo "SKIP: $FILE already has $EXPECTED bytes"
        continue
    fi
    if [ -e "$WORK" ]; then
        echo "ERROR: Incomplete working file exists: $WORK"
        echo "Preserve it for diagnosis; do not overwrite it automatically."
        exit 1
    fi

    echo "BACKUP: mtd$i ($NAME), expected $EXPECTED bytes"
    if [ "$i" = 2 ] || [ "$i" = 3 ] || [ "$i" = 5 ] || [ "$i" = 6 ]; then
        if ! dd if="/dev/mtd${i}ro" of="$WORK" bs=4096 > "$LOG" 2>&1; then
            echo "ERROR: Direct read failed: mtd$i. Review $LOG"
            exit 1
        fi
    else
        if ! nanddump --bb=padbad --omitoob -f "$WORK" "/dev/mtd$i" > "$LOG" 2>&1; then
            echo "ERROR: NAND dump failed: mtd$i. Review $LOG"
```

Script lines 67-86 of 86

```
            exit 1
        fi
    fi

    ACTUAL=$(wc -c < "$WORK")
    if [ "$ACTUAL" -ne "$EXPECTED" ]; then
        echo "ERROR: Wrong size for mtd$i: got $ACTUAL, expected $EXPECTED"
        exit 1
    fi
    mv "$WORK" "$FILE"
    echo "SUCCESS: mtd$i ($NAME), $ACTUAL bytes"
done

cd "$OUT"
sha256sum mtd*.bin > SHA256SUMS
sha256sum -c SHA256SUMS
sync

echo 'Completed. Check that all 11 files passed SHA-256 validation.'
echo 'Keep the USB mounted until all writes are flushed and checks succeed.'
```

Appendix B. Evidence recorded in the successful session

- Root shell: uid 0 on hostname modem.
- Kernel: Linux 4.4.115; SoC: EcoNet EN7580; boot argument ubi.mtd=system.
- NAND: 262,144-byte eraseblocks, 4,096-byte pages, 128-byte OOB.
- USB: /dev/sda1, FAT, mounted at /mnt/sda1; 32.09 GB drive.
- mtd0 and mtd1: 2,097,152 bytes each; no reported ECC errors.
- mtd2: nanddump failed on partial eraseblock 9; dd via /dev/mtd2ro recovered exactly 2,590,920 bytes.
- mtd3, mtd5, and mtd6: direct MTD character reads succeeded.
- mtd4, mtd7, mtd8, mtd9, and mtd10: nanddump completed without reported ECC errors or bad blocks.
- All 11 SHA-256 verification results were OK; final sync completed.
- Normal maintenance settings were restored and optics were reported Up/GOOD.
- The raw images remained on the USB drive and were not uploaded.

Source basis: commands, diagnostics, partition listing, and completion status provided during the 7 November 2026 troubleshooting session. This guide does not claim an independent review of the user-local diagnostic log files.

Appendix C. Offline firmware findings

Results supplied after the 11-image NAND backup, 7 November 2026.

Validated checks and image structure

All 11 saved images passed SHA-256 validation on the offline copy.
The four unaligned MTD regions were read through the MTD read-only character devices.
Both combined tclinux images were checked byte-for-byte against the separate
kernel and SquashFS images; the component ranges matched exactly.

Image slot	Firmware version	Kernel offset to SquashFS
Primary (mtd4)	CKT001-02.02.39.00	0x2788C8
Secondary (mtd7)	CKT002-02.04.56.00	0x276AD5

SquashFS metadata

Primary rootfs (mtd3): 22,382,502 bytes compressed; 2,782 inodes;
SquashFS v4, gzip/zlib; created 15 Mar 2022 at 13:05:05.
Secondary rootfs (mtd6): 36,356,060 bytes compressed; 3,107 inodes;
SquashFS v4, gzip/zlib; created 30 Oct 2024 at 13:45:34.

Differences relevant to the XGS-PON investigation

Different between slots: xpon.ko, xpon_10g.ko, xponmap.ko,
the OMCI daemon, and the platform manager.
Byte-identical between slots: gponmgr and gpon_flow.ko.
This directs offline analysis toward the changed XPON and OMCI components.
It does not by itself reveal the ONU authentication method or secrets.

Suggested offline, read-only comparison (Linux/WSL)

```
mkdir -p analysis/primary analysis/secondary
unsquashfs -d analysis/primary mtd3_rootfs.bin
unsquashfs -d analysis/secondary mtd6_rootfs_slave.bin
diff -qr analysis/primary analysis/secondary > analysis/diff.txt
```

INTERPRETATION / SECURITY

The running 2024 kernel build is consistent with the secondary image, but it
does not independently prove which boot slot is active. Confirm the boot selection
before making that claim. Extracted files may contain private credentials.
Keep the image files and all extracted configuration data in private storage.